L PLATE LOGGER

Freddy Gaffey

Table of Contents

Section 1 - Identifying and Defining (20 Marks) 2

1.1 — Client Scenario (3 Marks) 2

1.2 — Environmental Specification (3 Marks).....4

1.3 — List of Objectives (3 Marks).....11

1.4 — Performance Criteria (3 Marks).....14

1.5 — IPO (Input-Process-Output) (3 Marks) 15

1.6 — Gantt Chart (5 Marks).....16

Milestone 1: This should be completed by term 1, week 3. .....**Error! Bookmark not defined.**

Section 2 - Research and Planning (10 Marks) ... 20

2.1 — Research (5 Marks) 20

2.2 — Data Flow Diagram (DFD) (10 Marks) 31

2.3 — Storyboard (5 Marks).....34

2.4 — Pseudocode (10 Marks).....38

Milestone 2: This should be completed by term 1, week 6. .....**Error! Bookmark not defined.**

Addendum

All the charts were made on a tool called drawIO. I have included images of them in this document but if for some reason they are not visible, hard to read or follow, I have also included high-resolution images all relevant drawings and also the original drawIO document.
3bb3946c91e18b0b4468faf7638f5a73ab148972 is the commit with the documentation
Look in

documentation*
documentation\drawings*

This copy of the documentation is only valid as of 21/03/2026 as the actual master is on my CGS OneDrive so if I ask for feedback, please go to master_documentaion.docx I am sorry for the inconvenience, but it take 20 minutes for me to open OneDrive with many incognito widows sign in sign out desktop to webapps so this is the only way that I can not waste class time. Thanks for understanding!!

Also after writing this document, I have made a decision to switch for a PWA design to a capacitor app. This is a framework that lets you run a web app natively this will allow me to have a much cleaner experience on IOS as Apple doesn't let a PWA access the BLE api. But as the switch was made recently, I didn't rewrite the entire documentation so the document is written as if I was going to use a PWA but there might have been some small slipups towards the end.
So a if you go back in my git history, you will find lots of references to PWA then I restarted focusing on using Capacitor.

Section 1 - Identifying and Defining (20 Marks)

1.1 — Client Scenario (3 Marks)

Define a clear scenario based on the client's needs, including the application's purpose, target audience, and primary goals. Detail the client's requirements and provide a narrative that encapsulates the problem that the application aims to solve. Explain the overall vision and the expected impact of the application.

Purpose: The need is to make a service that automates the process of logging their hours as a learner driver as well as the hardware to support that. Additionally, it simplifies the process of changing between proficient and learner drivers by controlling an automated flipping plate.

Target Audience: Learner drivers and their fully licensed supervising drivers.

Primary goals: The primary goal is to make an app/website that **integrates with physical L plates**, **logs hours**, **and provides you with data analytics**.

The story: Currently, learner drivers often break the law by failing to fill out their logbooks correctly due to forgetfulness. Because the process takes 3 minutes to fill

in, but they are under pressure, skip it, ESPecially when parents are running late. You might even forget (intentionally or not) to put the L plates on. This is bad because it results in less qualified and capable drivers on the road. My MDP will solve this by making this whole process more efficient with the app and compatible hardware. The process of logging a drive should take very minimal time to complete; therefore, the logbook will always be filled in. The client's workflow should be to open their phone and press start, maybe indicate who the supervising driver is and then at the end, press stop, or it will automatically stop when out of Bluetooth range.

System environment:
The project relies on a range of underlying technologies, which adds complexity but is necessary to meet the full scope of the project. There are two ESP's one is simply a Bluetooth slave to the master. The master will have a GPS and an accelerometer on board. The GPS will run constant interpolation to figure out what the odometer is, to keep that in sync, as well as broadcast a website. This website will allow the user to start a trip and end one, as well as some other features. Once the trip is logged, it will be saved to an SD card on board. The master ESP32 will broadcast a Wi-Fi network if I can't connect to a saved home Wi-Fi network. If it can connect to the home Wi-Fi network, it will, and it will send it to the server. Once the user is redirected back to the PWA the redirect request will contain the essential data from the trip so it can update the PWA dashboard.

Objectives/features:
The objectives and features for my solution

- **Log trips** GPS, and accelerometer data
- In the GUI, have a way to view **trip analytics,** such as a map where colour represents speed
- Due to **mixed content blocking,** which is where a https website can't make a request to an http site, this is what is being served by the ESP32, so therefore I need **two websites** one that controls the hardware and the L plate and also one that is the main PWA. This integration should be as seamless as possible. Additionally, further technologies and platform research may yield an ultimate solution that is less complicated.
- The ability to **track the day and night hours** is probably displayed by 3 pi chats, one for each and one for the total
- The whole system can **work offline**, except for uploading, which requires the internet
- An AI **chatbot** that has RAG for understanding road rules
- **RBAs** that allow parents to approve drives once completed
- The app will be **optimised** around the **speed to start a drive**. Therefore, the app starts with a BRB (big red button). This is a UI design term that makes the

app more efficient to use, allowing the user to go to a new page that allows them to start a drive or view drives

1.2 — Environmental Specification (3 Marks)

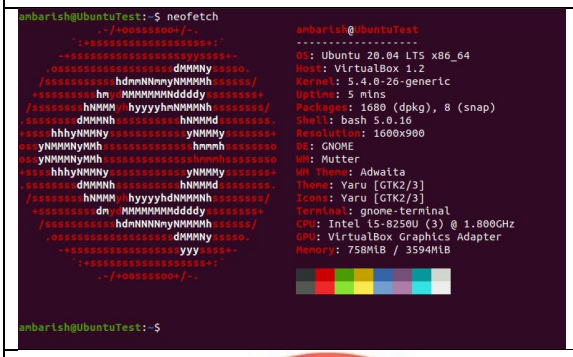
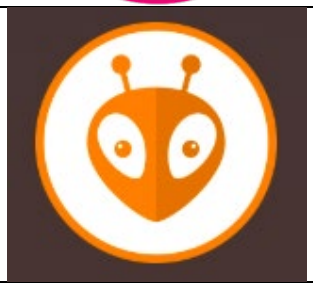
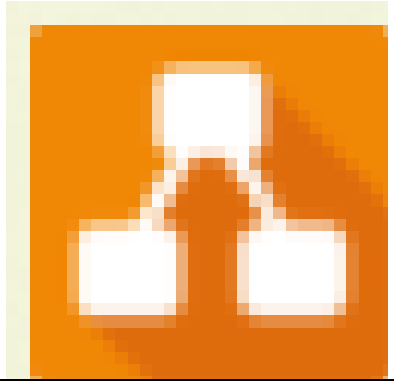
Outline the technical environment in which the application will operate. This includes hardware, software, network specifications, and other relevant environmental factors. Describe the operating systems, development tools, servers, and other infrastructure components necessary for the application.

Hardware

Property	Value
OS Name	Microsoft Windows 11 Home or Ubuntu (dual boot)
Version	10.0.26200 Build 26200
OS Manufacturer	Microsoft Corporation
System Name	*[REDACTED]*
System Manufacturer	HP
System Model	HP Envy x360 2-in-1 Laptop 14-fc0xxx
System Type	x64-based PC
System SKU	*[REDACTED]*
Processor	Intel Core Ultra 7 155U, 2100 Mhz, 12 Cores, 14 Logical Processors
BIOS Version/Date	Insyde F.05, 6/11/2024
SMBIOS Version	3.6
Embedded Controller Version	99.27
BIOS Mode	UEFI
BaseBoard Manufacturer	HP
BaseBoard Product	8C65
BaseBoard Version	99.27
Platform Role	Mobile
Secure Boot State	On
PCR7 Configuration	Elevation Required to View
Windows Directory	C:\windows
System Directory	C:\windows\system32
Boot Device	\Device\HarddiskVolume1

Locale	United States
Hardware Abstraction Layer	Version = 10.0.26100.1
User Name	*[REDACTED]*
Time Zone	AUS Eastern Daylight Time
Installed Physical Memory (RAM)	16.0 GB
Total Physical Memory	15.5 GB
Available Physical Memory	857 MB
Total Virtual Memory	39.7 GB
Available Virtual Memory	3.02 GB
Page File Space	24.2 GB
Page File	C:\pagefile.sys
Kernel DMA Protection	On
Virtualization-based Security	Running
VBS Required Security Properties	Base Virtualization Support
VBS Available Security Properties	Base Virtualization Support, Secure Boot, DMA Protection, UEFI Code Readonly, SMM Security Mitigations 1.0, Mode Based Execution Control, APIC Virtualization
VBS Services Configured	Hypervisor enforced Code Integrity
VBS Services Running	Hypervisor enforced Code Integrity
App Control for Business Policy	Enforced
App Control for Business User Mode Policy	Off
Automatic Device Encryption Support	Elevation Required to View
Hypervisor	Detected (Hyper-V features hidden)

Software ☑

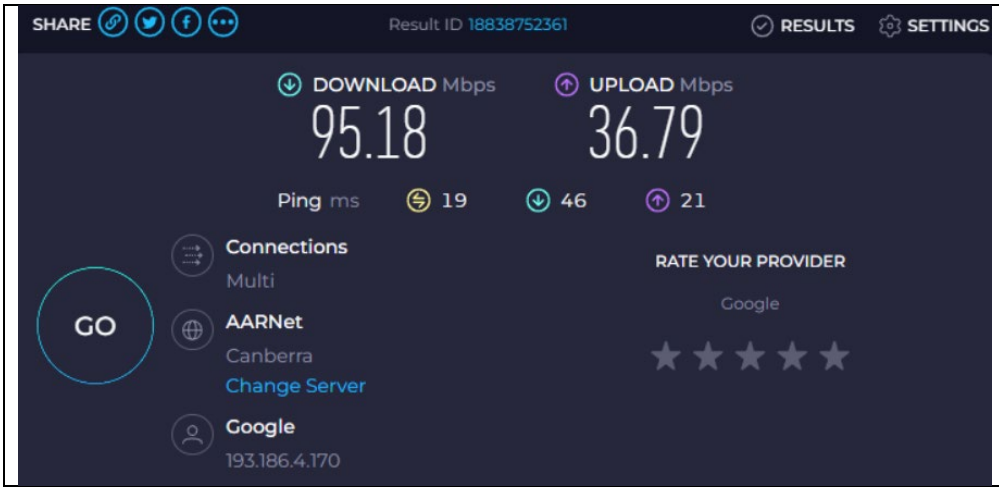
Image	Platform	Specification
	**Ubuntu 20.04 LTS (Linux Server)**	Linux Kernel 5.4+ widely compatible
	Google chrome	Any currently supported version
	Mozilla Firefox	Any currently supported version
	Platform IO	https://github.com/platformio/platformio-vscode-ide/releases/tag/v3.3.4
	Draw IO	The latest >=

	ngrok	This lets me serve my website as https for testing
	Apple	iPhone and mac on a browser
	android	Phone and iPad
	Live server	Latest version published on VS Code app repository

Network

Image	Part	Specification
	gateway	Compact 2.5G Cloud Gateway with 30+ UniFi device / 300+ client support, 2.3 Gbps IPS routing, and selectable NVR storage.
	Switch	Ubiquiti UniFi USW-PRO-48-POE Managed 48 Port POE Gigabit Switch

	Access point	U7 Lite Compact, ceiling-mounted Wi-Fi 7 AP with 4 spatial streams and 2.5 GbE uplink.
	Wi-Fi config	

 SHARE Result ID 18838752361 RESULTS SETTINGS DOWNLOAD Mbps 95.18 UPLOAD Mbps 36.79 Ping ms 19 46 21 Connections Multi GO AARNet Canberra Change Server Google 193.186.4.170 RATE YOUR PROVIDER Google	Speed	

1.3 — List of Objectives (3 Marks)

Enumerate the specific objectives that the application aims to achieve. These should be clear, measurable goals that directly address the client's needs. The objectives could include functionality, user engagement, performance targets, and other relevant criteria.

Essential Features
Please note that all the features that are labelled as 10 are need as part to the MVP.

Feature	Priority 1-10	Description
1. Login	10	Provide the user the ability to use pre-saved credentials to access protected areas of the system
1.1 - Login window	10	Provide users with a screen which allows them to authenticate with the program. If a user tries to access a restricted location, redirect them to the login
1.2 - Username	10	User can create a unique username Must provide username at login If not found prompt user to sign-up
1.3- password	10	User uses a password which to authenticate their account Password must be 8 characters and contain numbers and symbols
2 – Data logs from the plate	8	This data is saved from onboard sensors and then transmit up to the cloud database
2.1 – Accelerometer data	5	This data will be saved at a rate that will be determined later and then sent up to the cloud. The purpose of this data is so that it is help provide analytics the user about driving style on things like braking or acceleration.

2.2 – GPS data	8	This is essential because this data is the way that the odometer is determined and also the trip distance.
2.3 – speed data (extrapolated from GPS in backend)	6	This is data that tracks the speed in the vehicle also provides analytics on if you were speeding* this is a potential feature that might not be technically feasible to get if speeding
3 - plate battery data	6	Data on if the L plate is low on battery so that it can be charged
4.1 - starting drive	10	This must be really easy thing to do it must take under 10 seconds, it must also work off line
4.2 - stopping drive	10	This must be really easy thing to do it must take under 10 seconds, it must also work offline
5 - The parent approves the drive	9	A parent/supervising driver must be able to approve a drive easily
6 - viewing the drives and being able to fill in the log book from that	10	The ability to view the logs in a format that is easy to transcribe in to the physical log book.
7 - Ai chat bot	5	The ai chat bot that can give advise on the relevant laws
8 - Dashboard to view the driving stats	8	The dashboard that will let the user view their historical drives
9 - Drive detail view	6	Can select a drive and view the map and colour with acceleration
10 - Day and night hours and drives separation	9	The day drives are done sand at night are separated by colour on the dashboard

Non-Essential Features

Feature	Priority	Description
Work offline PWA or equivalent	6	Australia does not have mobile data for the whole country so the l plate system must be able to work off line.
Work on iOS, Android, Windows, mac Linux(desktop)	8	The logging is done on a phone and the viewing of hours is done on a laptop or phone
Have 95% uptime	7	Since the app can work off line for the primary role (logging driving) some minimal downtime is permissible because all they happens they can't view their logbook so if late at night, then the down time is not a problem
Be fast to start logging a drive	9.5	This is the problem that I am solving with he whole app so it is essential

1.4 — Performance Criteria (3 Marks)

Specify the application's performance criteria, including speed, reliability, scalability, and user satisfaction benchmarks. Define how performance will be measured and the application's standards.

Speed
The L plate controller (hosted by the ESP 32) should be fast and rESPonsive, getting a score of >85 on the Google Lighthouse.

The time for an API call to execute, including DB read/write time, should be under 200 ms. This will be measured with a baseline load of 10 users, but wrapping each call with a Python “time. performance” call and saving that.

Reliability
The main website (PWA) should have an uptime of 95% this is much lower than other websites. However, this is acceptable because the website is a PWA and the critical functionality is all on the client side, unlike most other websites. To assess my success, I will make a VPS that will ping my website to see if it is still running then I will measure the average uptime over the month.

Scalability
The app should be scalable and support 1000 users concurrently. I will use a make a python script that will make randomised api calls.

User satisfaction benchmarks
To assess the success of the website, I will use DB analytics to calculate user retention. This should be 80%. If this is on the app store it should maintain 4/5 stars for the first month.

Design of UI
I will remove the text field as they are slow to fill, so I will try to fill them with automated data drawn from OBD scanners and GPS readings.

In order to solve the problem, the user must be able to start a drive in 10 seconds from opening the app.

1.5 — IPO (Input-Process-Output) (3 Marks)

Develop an IPO model that outlines the application's inputs, processes, and outputs.

Input	Process	Output
Accelerometer GPS Time SD Start time End time	Write data to sd card in a csv file and then uploading it to the cloud when on Wi-Fi	Sent log to the cloud
Password Username	Check username and password in the db	Authenticate the user return a jwt
Approve(trip) or Disapprove(trip)	Change the trip approval state to approve or disapprove	A db modification change in UI
SD name Start odo Start time	Save the data to the right file	the log file
End time End odo Log file	Do some data processing to improve efficiency Close file	Send the log to the cloud
Accel data Gps data	Update the body of log file with the data	The data added to the log file
User prompt Rag document Past chat history	The llm black box	Answer to the question/prompt
First name Last name Email State Password	Save those files to the database	Return JWT

1.6 — Gantt Chart (5 Marks)

Create a Gantt chart to outline the project timeline. The chart should include all major milestones, tasks, and deadlines. It should also visually represent the project schedule, showing each task's start and end dates and the dependencies between tasks.

Task ID	Task Name	Duration	W1 26/01	W2 02/02	W3 09/02	W4 16/02	W5 23/02	W6 02/03
1	**SECTION 1: IDENTIFYING & DEFINING**	28						
1.1	Client Scenario Development	7						
1.2	Environmental Specification	14						
1.3	List of Objectives (Functional)	7						
1.4	List of Objectives (Non-Functional)	7						
1.5	Performance Criteria	14						
1.6	IPO Charts	7						
1.7	Gantt Chart Creation	7						
M1	**MILESTONE 1: Section 1 Complete (Week 3)**	7			@			

Task ID	Task Name	Duration	W1 26/01	W2 02/02	W3 09/02	W4 16/02	W5 23/02	W6 02/03	W7 09/03	W8 16/03	W9 23/03
2	**SECTION 2: RESEARCH & PLANNING**	35									
2.1	Market & Competitor Research	14									
2.2	Technology Stack Research	14									
2.3	Data Flow Diagrams (Login)	7									
2.4	Data Flow Diagrams (Trip Logging)	7									
2.5	Data Flow Diagrams (AI Chatbot)	7									
2.6	Storyboard/Wireframes	14									
2.7	Pseudocode - Authentication	7									
2.8	Pseudocode - Trip Processing	7									
2.9	Pseudocode - Parent Approval	7									
M2	**MILESTONE 2: Section 2 Complete (Week 6)**	7								@	

Task ID	Task Name	Duration	W1 26/01	W7 09/03	W8 16/03	W9 23/03	W10 30/03	W11 06/04	W12 13/04	W13 20/04	W14 27/04	W15 04/05	W16 11/05	W17 18/05	W18 25/05	W19 01/06	W20 08/06	W21 15/06	W22 22/06	W23 29/06
3	**SECTION 3: DEVELOPMENT & PRODUCTION**	98																		
3.1	Backend Setup (Flask/SQLite)	14																		
3.2	User Authentication System	14																		
3.3	Database Schema Implementation	14																		
3.4	ESP32 Firmware - GPS Module	14																		
3.5	ESP32 Firmware - Accelerometer	14																		
3.6	ESP32 Firmware - SD Card Logging	14																		
3.7	Trip Upload API Endpoints	14																		
3.8	React Frontend - Logger Interface	21																		
3.9	React Frontend - Dashboard	14																		
3.10	Parent Approval Workflow	14																		
3.11	PWA Implementation (Service Workers)	14																		
3.12	AI Chatbot - RAG Setup	14																		
3.13	AI Chatbot - NSW Laws Integration	14																		
3.14	Day/Night Hour Tracking	7																		
3.15	Progress Visualisation Charts	14																		
3.16	Logbook Maintenance	98																		
3.17	Version Control (Git)	98																		
M3	**MILESTONE 3: Development Complete (Week 22)**	7																	@	

Task ID	Task Name	Duration	W20 08/06	W21 15/06	W22 22/06	W23 29/06	W24 06/07	W25 13/07	W26 20/07
4	**SECTION 4: TESTING & MAINTAINING**	35							
4.1	Acceptance Testing	14							
4.2	Load Testing	14							
4.3	User Survey Design	7							
4.4	User Survey Distribution	14							
4.5	Bug Fixes & Refinements	14							
4.6	Video Demonstration	14							
4.7	Final Documentation Review	7							
M4	**MILESTONE 4: PROJECT DUE (21 July 2026)**	7							@

LEGEND:
Section 1: Identifying & Defining
Section 2: Research & Planning
Section 3: Development & Production
Section 4: Testing & Maintaining
Milestone

Section 2 - Research and Planning (10 Marks)

2.1 — Research (5 Marks)

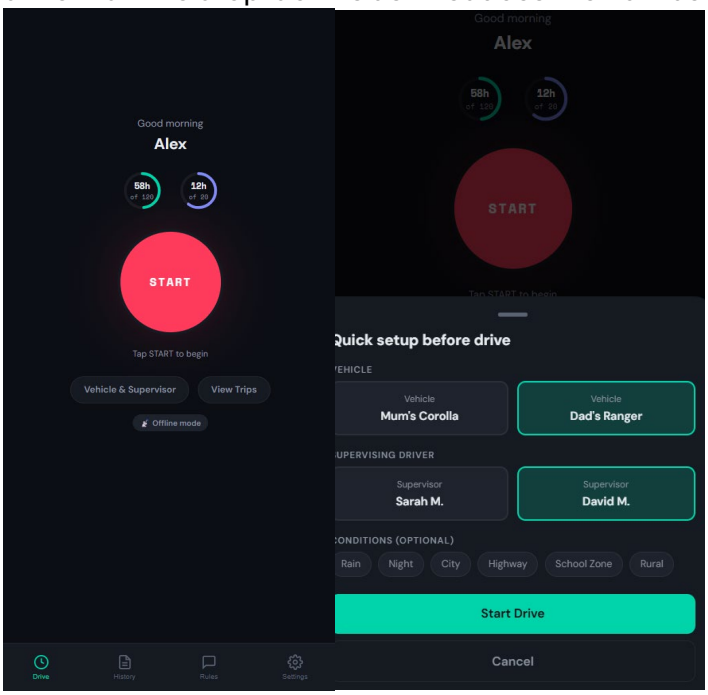
Conduct thorough research to gather information relevant to the project. This should include market analysis, user research, competitor analysis, and other pertinent data. Summarise the findings and explain how they will inform the project's development.

Introduction

Extensive research was required for this project to ensure that the decisions I made were informed and evidence-based. An investigation of the components was needed to accelerate the development of the UI. An exploration was conducted on the design of the UI. AI was employed to generate multiple different designs. I then asked users for which designs they preferred and why. This allowed me to help refine my decisions. To choose this web stack for my project, I reviewed a number of the options that are out there evaluated and then I chose the one which best suited my needs.

UI design

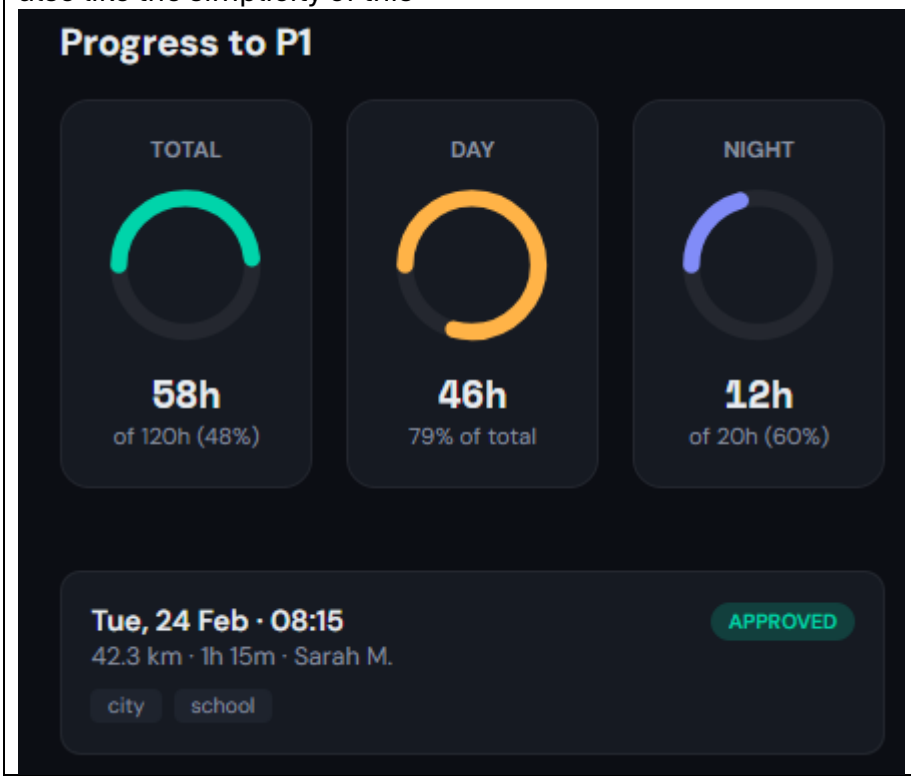
Having surveyed 10 of my peers, I found that 8 of the 10 surveyed preferred one design. The overwhelming favourite was the single circular button instead of the rectangular start button, as shown below. This is a faster and clearer way to start a drive than the drop-downs as it reduces the number of clicks.



3/5 of the people I asked said that they preferred pi chart over the any of the other way displaying data they liked the this the one thay pointed to when they said that they liked this



As it cleanly uses the pi charts to show useful information in a clean way and they also like the simplicity of this



Market analysis

Demographics

Non-fully licensed drivers and anyone who shares a car with them. This include the L-plate divers leaning in parents cars and P-plate drivers who share with their parents.

Where do they live?

In every state of Australia.

Pain points

This will appear to the user because it is better than the other options in the following ways

- Integrated logbook filling out
- Easier to switch from displaying plate to not (don't have to get out of car)
- Helps you become a better driver because of the ai teacher built into the app
- Having to type in the ODO each time Especially when in a rush

Patterns and trends, how do they already solve the problem

People either use the paper logbooks or they use the apps like roadtrip (the official NSW app), but the problem is that these apps are comparatively slow and hard to log a trip fast

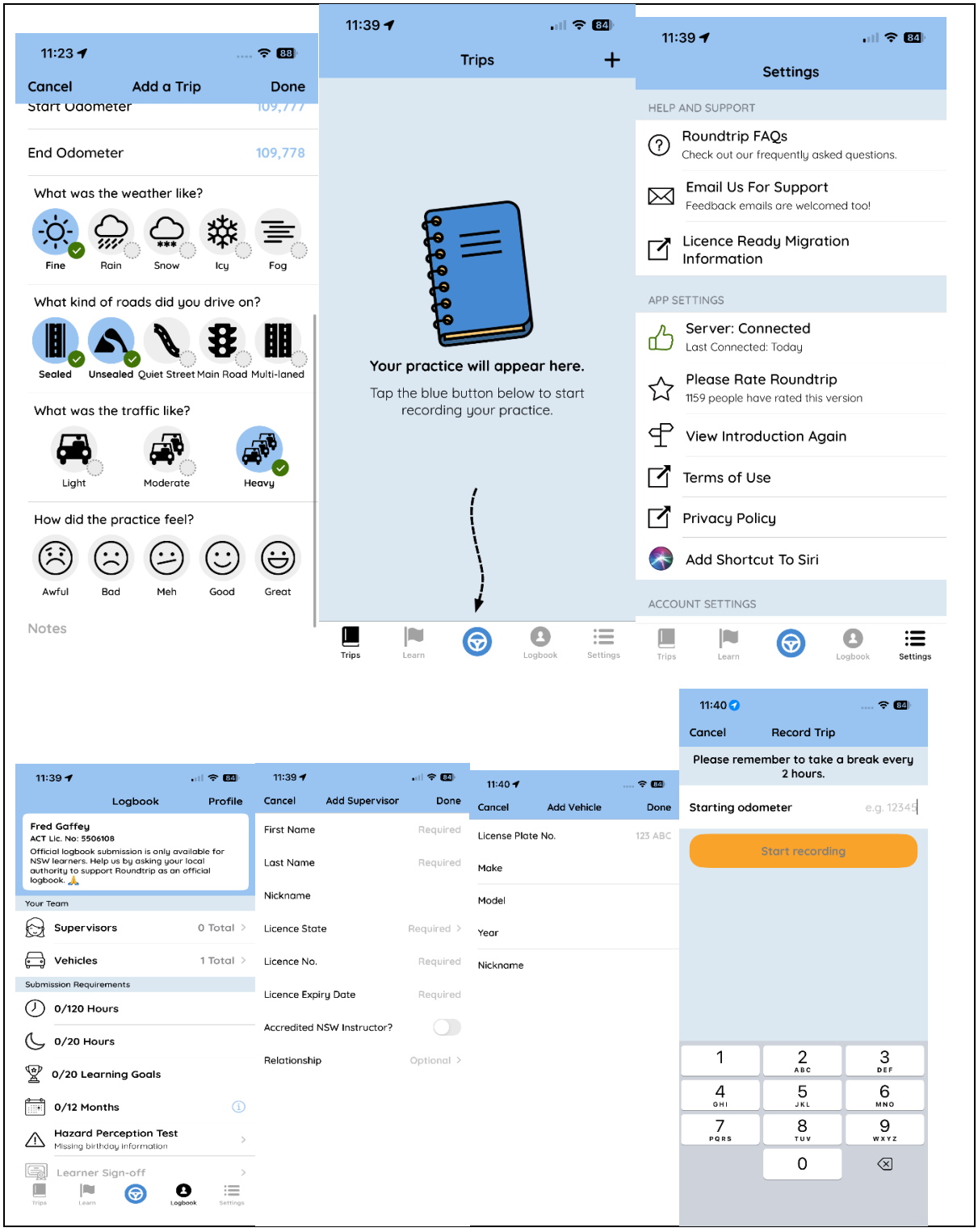
Challenges entering the industry

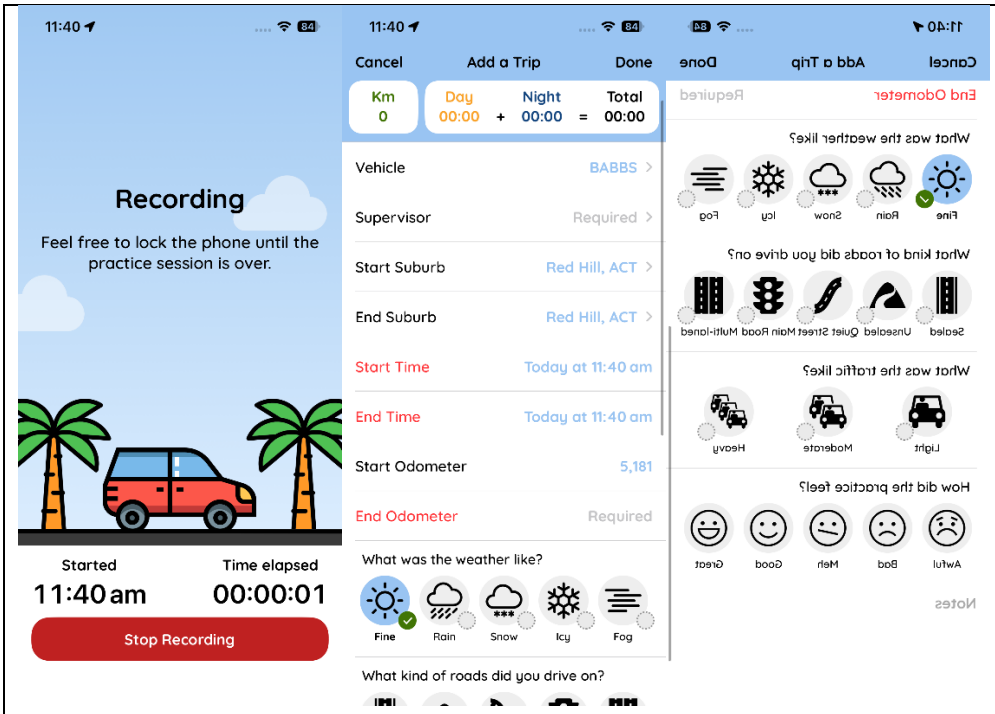
Getting approval from each state to be allowed to make this an official app is a challenge, but given the scope of this project, it is not necessary.

How is the industry growing

The industry is relatively stable and is not hard to saturate and as there is ever more learner drivers as the old ones get their P's and new ones come of age.

Competitor Analysis


11:23
Cancel
Add a Trip
Done
End Odometer
109,778
What was the weather like?
Fine
Rain
Snow
Icy
Fog
What kind of roads did you drive on?
Sealed
Unsealed
Quiet Street
Main Road
Multi-laned
What was the traffic like?
Light
Moderate
Heavy
How did the practice feel?
Awful
Bad
Meh
Good
Great
Notes
11:39
Trips
Your practice will appear here.
Tap the blue button below to start recording your practice.
Trips
Learn
Logbook
Settings
11:39
Settings
HELP AND SUPPORT
Roundtrip FAQs
Check out our frequently asked questions.
Email Us For Support
Feedback emails are welcomed too!
Licence Ready Migration Information
APP SETTINGS
Server: Connected
Last Connected: Today
Please Rate Roundtrip
1159 people have rated this version
View Introduction Again
Terms of Use
Privacy Policy
Add Shortcut To Siri
ACCOUNT SETTINGS
11:39
Logbook
Profile
Fred Gaffey
ACT Lic. No: 5506108
Official logbook submission is only available for NSW learners. Help us by asking your local authority to support Roundtrip as an official logbook.
Your Team
Supervisors
0 Total
Vehicles
1 Total
Submission Requirements
0/120 Hours
0/20 Hours
0/20 Learning Goals
0/12 Months
Hazard Perception Test
Missing birthday information
Learner Sign-off
11:39
Cancel
Add Supervisor
Done
First Name
Required
Last Name
Required
Nickname
Licence State
Required
Licence No.
Required
Licence Expiry Date
Required
Accredited NSW Instructor?
Relationship
Optional
11:40
Cancel
Add Vehicle
Done
License Plate No.
123 ABC
Make
Model
Year
Nickname
11:40
Cancel
Record Trip
Please remember to take a break every 2 hours.
Starting odometer
e.g. 12345
Start recording
1
2 ABC
3 DEF
4 GHI
5 JKL
6 MNO
7 PQRS
8 TUV
9 WXYZ
0



My first impressions of the NSW logbook app were generally positive. However, it was immediately apparent that the app's UI had some deficiencies. Examples of this include a slow and clunky user experience when creating a new drive and a cumbersome user interface with lots of fields.

To gather more objective feedback I surveyed students from two classes of DT (Design Technology) students at my school. The total number of students sampled were 10, where I walked them through the app and asked them to give a verbal commentary on what they liked and disliked. I did the same for my prototype designs. The students were then asked to rate each design against three criteria: aesthetics, efficiency and intuitiveness.

The results of the survey suggest that people thin that the NSW app had all the features that they wanted. However, while the layout was very aesthetically pleasing, it was deemed not very efficient.

The issues raised by those surveyed led me to implement improvements such as auto-filling the odometer field based on the car selected. Also, my graphics will look nicer, with better plots and map visualisation. I will similarly replace the learn feature of this app with an AI that has relevant RAG context on specific state rules and regulations.

Framework justification

I chose to use **Vue** because I liked the way that the docs read it is well supported by Capacitor, and my teacher recommended it to me. All these factors resulted in my choosing to go with Vue as my front-end framework.

I chose to use **Flask** because it was already familiar to be as I had used it in my AT1 it was Python-based and Python has lots of libraries and also Python has great libraries, and due to it being an interpreted language, it has a faster development cycle, allowing for more agile development.

I chose to use **JWT** as they are resistant to CSRF and they are also so easy to implement and they are made by Google.

I chose **SQLAlchemy** as it is very Pythonic as it is very OOP. This means that working with is will be easer as there is less time needed to context switch from one paradigm to another. Also the main reason I chose this is that the software engineering department at the Kings School recommended it as it was ease and a good choice for this assignment.

I chose to use **chart.js** as my data visualisation library, it has good community support, is has uses HTML5 canvases instead of SVGs, therefore better performance on mobile. But most importantly, it is easy to integrate into my existing stack of vue.

I chose to use **Leaflet.js** as my map visualisation tool because it is simple to use it is very lightweight, easy to integrate with vue and I have also used it before; therefore, it is easer to integrate with the exiting stack.

Conclusion

Based on this extensive research, I have decided to assemble a stack made of Flask, Vue and SQLAlchemy. For authentication, I will use JWT and a standard library to ensure reliable authentication. I will go with a UI similar to that from my prototypes. However, based on user feedback, I am looking to incorporate some of the more successful features from the NSW road trip app. Having seen many of the technologies I intend to implement are similar to those used in the roadtrip app has provided some reassurance that my choices are reasonable and have parallels with those used in industry.

Research log
https://www.youtube.com/watch?v=sFsRylCQblw
chrome lighthouse
Look at work box
https://hnPWA.com/ this is a website to help you pick a PWA framework
https://web.dev/
https://web.dev/learn/PWA/welcome?continue=https%3A%2F%2Fweb.dev%2Flearn%2FPWA%23article-https%3A%2F%2Fweb.dev%2Flearn%2FPWA%2Fwelcome
need to learn how to get the service worker to work
Web app
ESP 32 server:

First I wrote some very high level pseudo code for how I wanted the app to work. This helped me identify what liberties I needed.

- A database (or some way of storing tabula data)
- Ability to broadcast a wifi network
- Ability to connect to a wifi network and to send data to a cloud server
- Ability to pass and make json
- Ability to read a gps value
- Ability to read a accelerometer value

I chose the following library because of there ease of use and because thay are common choices for the problems I faced.

- WiFi.h
- ESPAsyncWebServerHTTP
- ArduinoJsonJSON
- LittleFSLocal
- HTTPClientSend

To lean about how to use them I asked Haiku 4.5 model to make me a list of the relevant docs and helpful tutorials I gave me the following list

“””

ESP32-C3 Drone Data Logger Project - Library Documentation

Core Libraries & Documentation

1. WiFi.h (Built-in ESP32 Core)

- Official Documentation: https://docs.ESPressif.com/projects/ESP-idf/en/latest/ESP32c3/api-reference/network/ESP_wifi.html
- Arduino WiFi Reference: https://www.arduino.cc/reference/en/libraries/wifi/

2. ESPAsyncWebServer

- GitHub Repository: https://github.com/me-no-dev/ESPAsyncWebServer
- Documentation: https://github.com/me-no-dev/ESPAsyncWebServer/wiki
- Installation Guide: https://randomnerdtutorials.com/ESP32-async-web-server/

3. ArduinoJson

- Official Site: https://arduinojson.org/
- Documentation: https://arduinojson.org/v6/api/
- Quick Start: https://arduinojson.org/v6/getting-started/

4. LittleFS (File System)

- ESP32 LittleFS Documentation: https://github.com/lorol/LITTLEFS
- Tutorial: https://randomnerdtutorials.com/ESP32-littlefs/

5. HTTPClient (for sending POST requests)

- Arduino HTTPClient Docs: https://www.arduino.cc/reference/en/libraries/httpclient/
- ESP32 Guide: https://randomnerdtutorials.com/ESP32-http-post-request/

6. Preferences.h (Built-in - Persistent Storage)

- Official Documentation: https://docs.ESPressif.com/projects/ESP-idf/en/latest/ESP32c3/api-reference/storage/nvs_flash.html
- Arduino Preferences Library Guide: https://randomnerdtutorials.com/ESP32-preferences-library/

- GitHub (ESP32 core): https://github.com/ESPressif/arduino-ESP32/blob/master/libraries/Preferences/src/Preferences.h

7. Time Management

- Time.h Reference: https://www.arduino.cc/reference/en/libraries/time/
- millis() and micros(): https://www.arduino.cc/reference/en/language/functions/time/millis/

For Analog Pin Reading (GPS/Accelerometer Simulation)

- ADC Reading: https://docs.ESPressif.com/projects/ESP-idf/en/latest/ESP32c3/api-reference/peripherals/adc.html
- Arduino analogRead(): https://www.arduino.cc/reference/en/language/functions/analog-io/analogread/

“””

I looked at the demos that the wifi libery provides and saw “simple web serve” by Tom Igoe this is a simple web server I used the code example to make a start_ap() function https://github.com/ESPressif/arduino-ESP32/blob/master/libraries/SD/examples/SD_Test/SD_Test.ino how to write to a sd card

L plate hardware:

THE PWA:
https://www.youtube.com/watch?v=L5O8uyX2cU8 looking at best web stacks

broke down choosing a tech stack
There is no one best tech stack, but there is a best tech stack for you and to determine it asses it with these following things.
https://www.youtube.com/watch?v=R5BwFCyV9Mk
What is a tech stack
Learning Curve
Ecosystem
Compatibility & Interoperability
Performance and Cost
Team Expertise
Future-proofing

I have now decided to use flask as I am very similar with python and I don’t need performance it has a bigger enough ecosystem

PWA stuff
https://blog.appmysite.com/what-is-a-progressive-web-app/
https://www.youtube.com/watch?v=wQgx5Pou44s
https://web.dev/learn/PWA
https://hnPWA.com/
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Tutorials

https://www.youtube.com/watch?v=sFsRylCQblw Progressive Web Apps in 100 Seconds // Build a PWA from Scratch : Fireship
https://developer.chrome.com/docs/workbox/
https://github.com/elegantapp/PWA-asset-generator

HOW TO COMMUNICATE TO THE ESP 32
https://stackoverflow.com/questions/53211319/how-can-i-make-an-http-request-from-an-https-website my fist idea was to have the ESP 32 host a server that acts as a API endpoint but this can't work because mixed content (having https and http) is unsafe so I need to find a way around that.

https://www.chatbase.co/ THIS IS FOR THE AI INTERGRATION
https://codezup.com/creating-a-progressive-web-app-with-flask-and-service-workers/ a tyotorial
Jinja2 template I need to learn about

https://github.com/TempeHS/Flask_PWA_Programming_For_The_Web_Task_Template THIS IS SO USEFULL IT IS DESIGEND FOR THE **HSC**
https://www.geeksforgeeks.org/python/python-property-decorator-property/ helpful understanding the @propidy decorator
https://codezup.com/implementing-authentication-in-flask-guide/ helped me implement auth
I need to use "from werkzeug.security import generate_password_hash, check_password_hash"
https://www.pythontutorial.net/python-oop/python-__repr__/ __repr__ method

THE BEST WAY TO LEAN ABOUT AUTH WAS THE FLASK DOCS THEMSELVES
https://flask-user.readthedocs.io/en/v0.5/authorization.html
https://github.com/maxcountryman/flask-login/blob/main/README.md
https://flask-login.readthedocs.io/en/latest/
I found about @app.get() and @app.post() this is a much nicer syntax then @app.rounte("/login" method=["post"])
https://github.com/pallets/flask/tree/main THE SOURCE CODE
---------- I have switched to capacitor ----------
I have decided to use
Vue
Flask
TS
Html
Css
Capacitor
Platform io
https://ionicframework.com/docs/components to find the vue components
this was for auth
https://ionicframework.com/docs/api/button
https://ionicframework.com/docs/api/input
##########

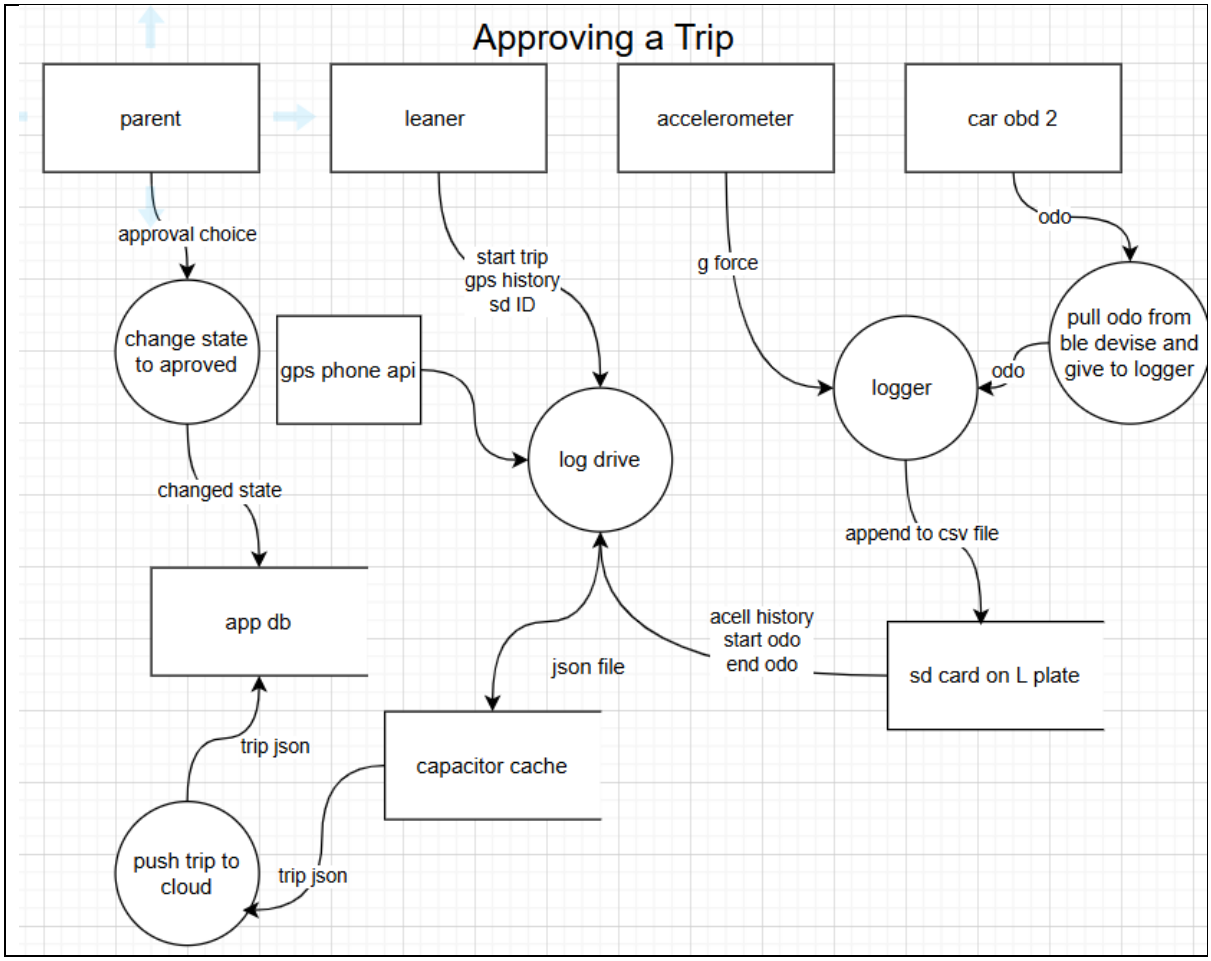
https://www.geeksforgeeks.org/python/sqlalchemy-core-multiple-tables/
I used this to try and work out how backpropagation works
https://manage.auth0.com/dashboard/au/dev-fofvk0oiugokkagr/applications/c9mN2JfRtDbllV52AcSghBBM7b3RTE31/quickstart/spa/vuejs to get auth zero to work

I had a long discussion with a llm to work how to do auth I decided to use jwt as it se I us seemed quite straightforward and also was resistant to csrf if used cookieys it would work well

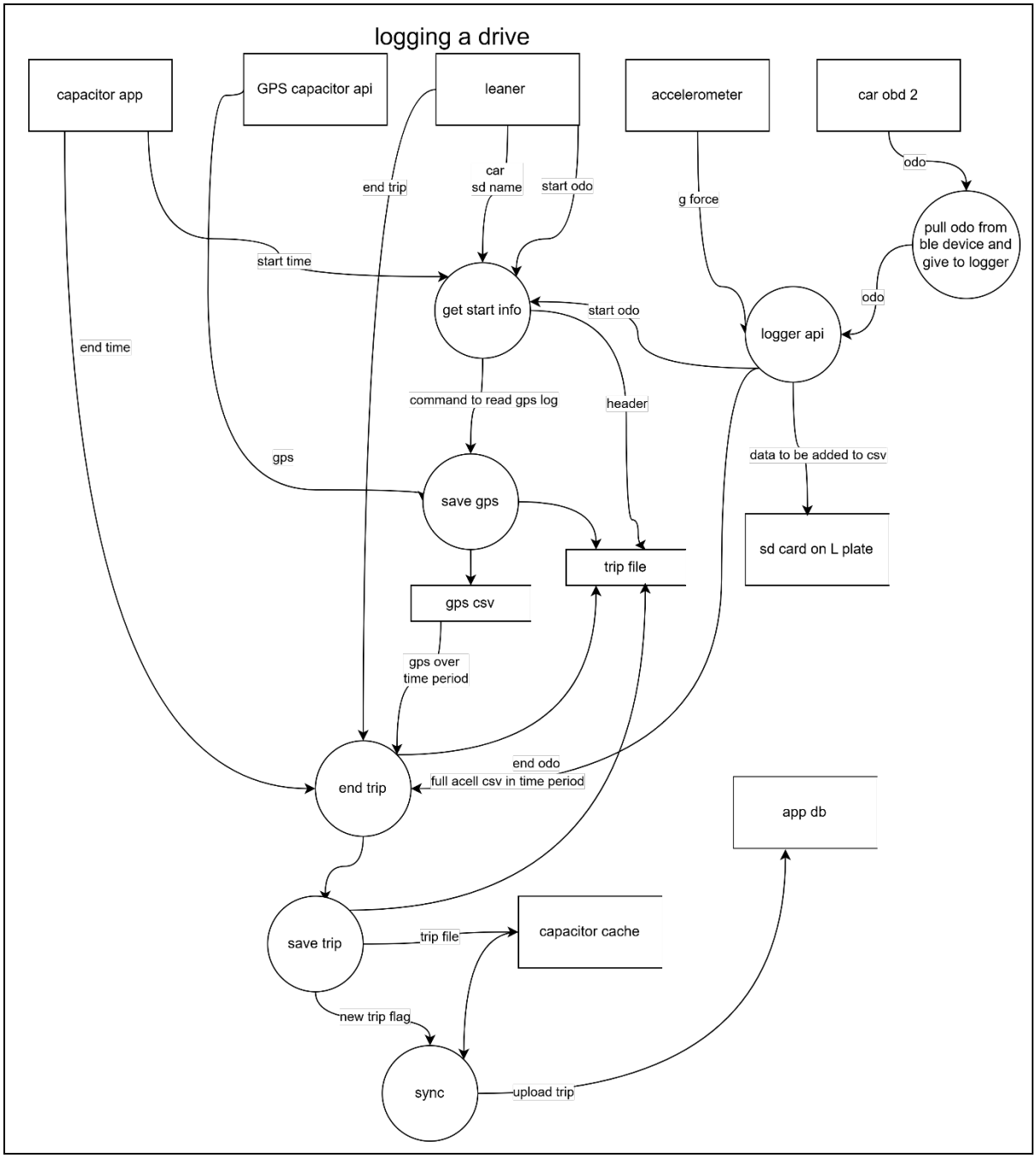
2.2 — Data Flow Diagram (DFD) (10 Marks)

Design a DFD for each feature of your project to illustrate how data will flow through the system. The DFD should show the interactions between inputs, processes, and outputs. Provide detailed diagrams and explanations for each feature.

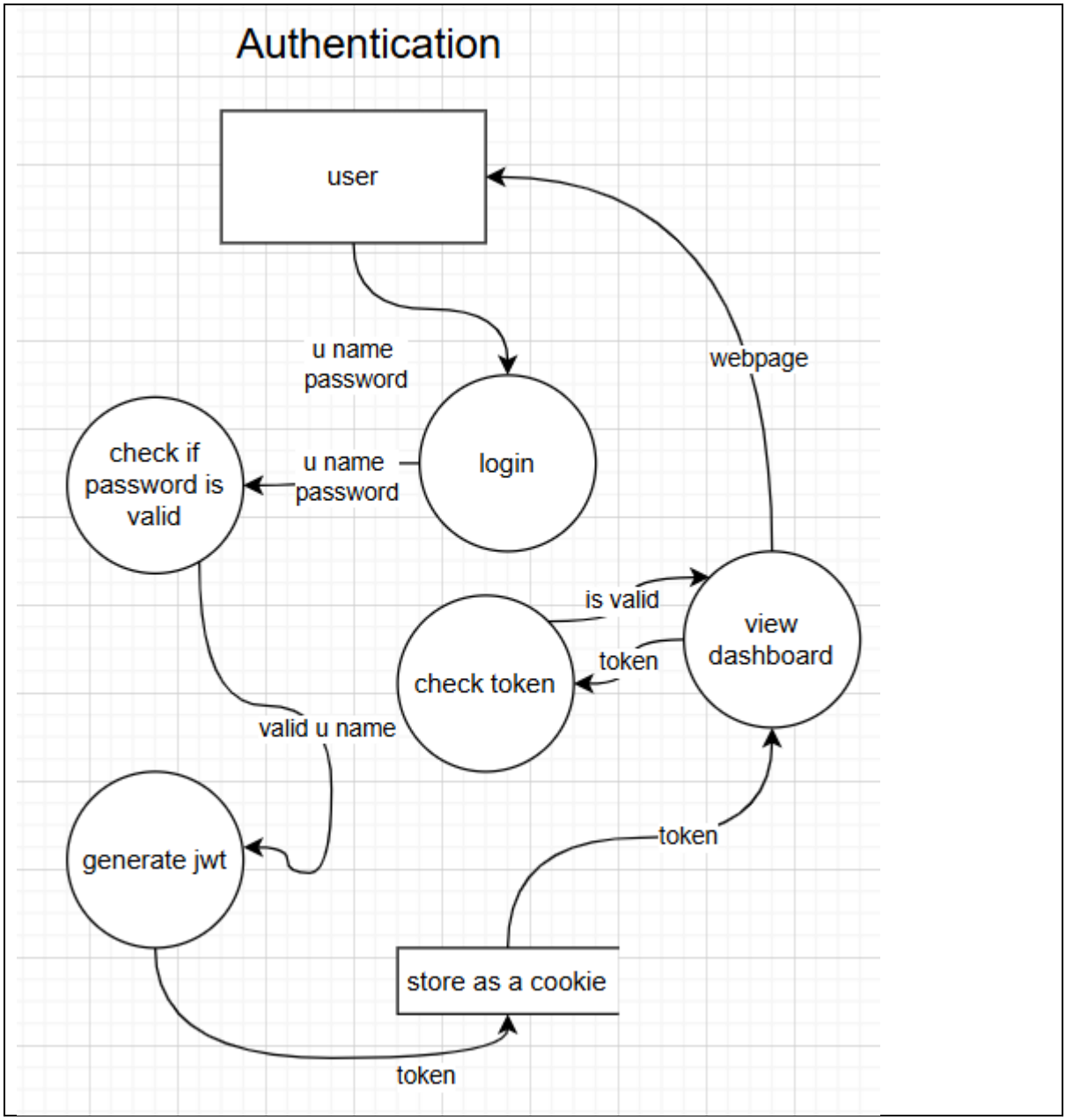
DFD 1



DFD 2

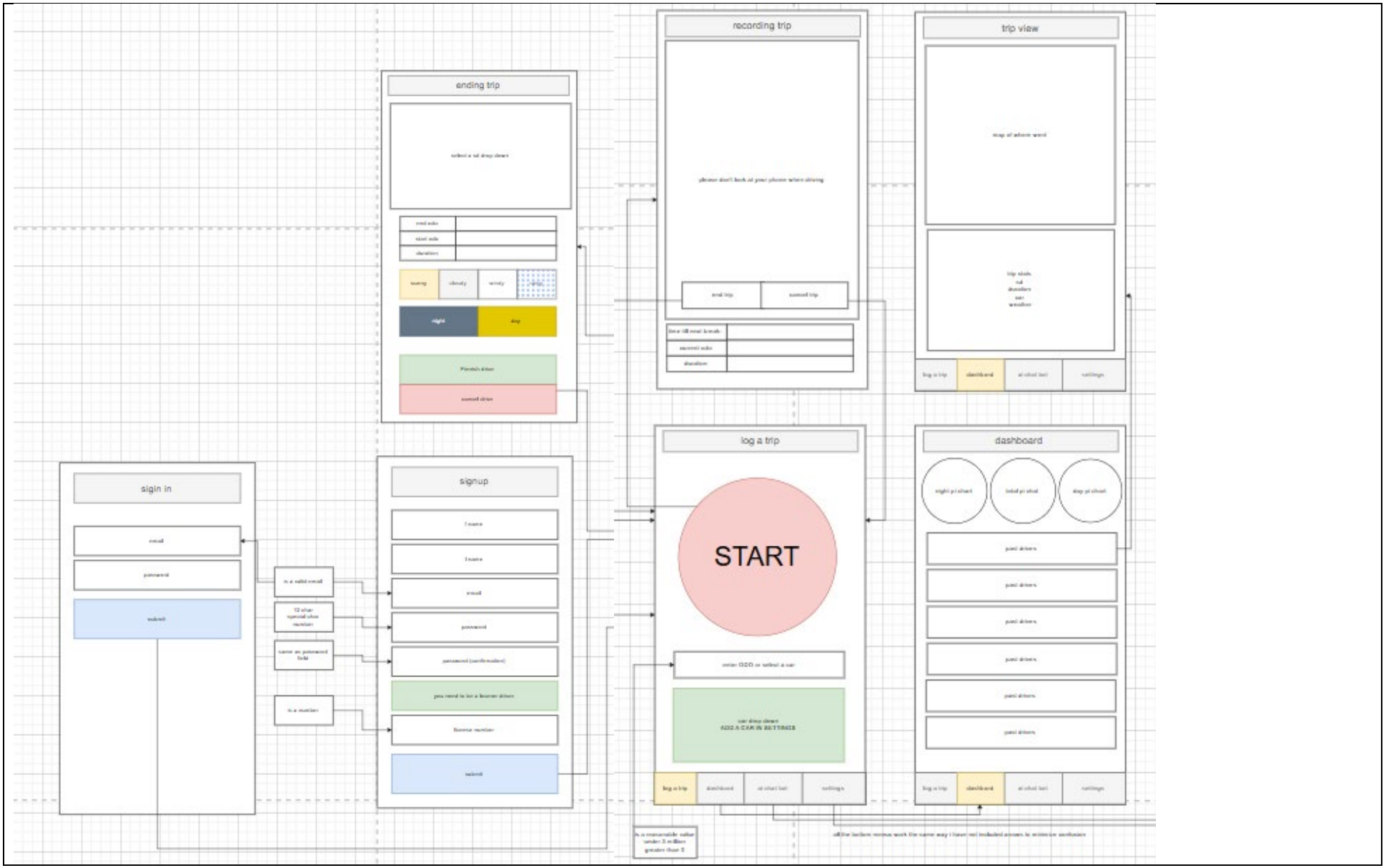

logging a drive
capacitor app
GPS capacitor api
leaner
accelerometer
car obd 2
odo
end trip
car
sd name
start odo
g force
pull odo from
ble device and
give to logger
start time
get start info
start odo
odo
logger api
end time
command to read gps log
header
gps
data to be added to csv
save gps
sd card on L plate
trip file
gps csv
gps over
time period
end odo
full acell csv in time period
end trip
app db
save trip
trip file
capacitor cache
new trip flag
sync
upload trip

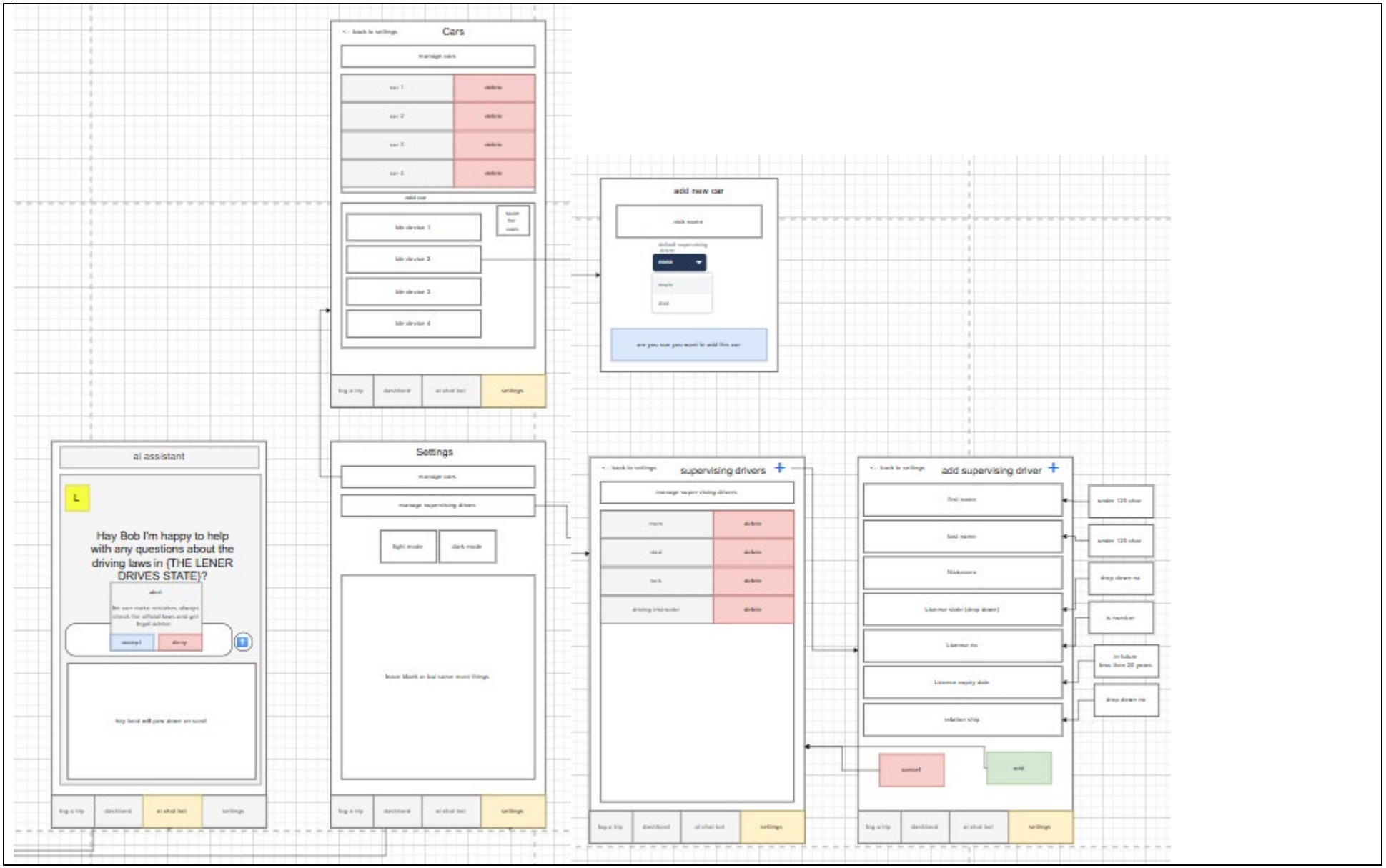
DFD 3


Authentication
user
u name
password
webpage
check if
password is
valid
u name
password
login
is valid
view
dashboard
token
check token
valid u name
generate jwt
token
store as a cookie
token

2.3 — Storyboard (5 Marks)

Create a storyboard to outline the user journey through the application. The storyboard should include key screens, user actions, and interactions. Provide a narrative to explain how users will navigate the application and achieve their goals.


ending trip
recording trip
trip view
sign in
signup
log a trip
START
dashboard


Cars
add new car
ai assistant
Hay Bob I'm happy to help with any questions about the driving laws in (THE LENER DRIVES STATE)?
Settings
supervising drivers
add supervising driver

Please see the PNG if you cant read this

2.4 — Pseudocode (10 Marks)

Write three pseudocode algorithms for critical functions and algorithms in the web application's backend. These pseudocode algorithms should provide a clear foundation for coding the application. Include detailed explanations and comments for each algorithm.

Algorithm 1

```
# this is the pseudocode on how the ai chat bot works
BEGIN main
  GET user_state
  GET system_prompt

  SET llm = start_chat(user_state)
  SET last_message = system_prompt
  SET continue_chat = True
  WHILE continue_chat = True
    GET continue_chat
    GET latest_message
    IF latest_message is not equal to last_message THEN
      send_to_llm(latest_message)
      SET last_message = latest_message
      IF ai_finnished = True THEN
        GET rESPonse_from_llm_api
        DISPLAY rESPonse_from_llm_api
      ENDIF
    ENDIF
  ENDWHILE
  main()
END main

BEGIN start_chat(user_state)
  GET system_prompt
  SET rag_document = get_rag_docment(user_state)
  SET llm = send_to_llm(rag_document,system_prompt)
  GET welcome_message
  DISPLAY welcome_message
  RETURN llm
END start_chat(user_state)
```

Algorithm 2

```
BEGIN
  # this psudocode is the core logic that will populate the log file with the neede data
  # the odb will save the acellromoter and also infomaton on the current odo
  SET gps_log_rate = 1/2
  SET acell_log_rate = 1/10
  connect_to_OBD()
  send_OBD_to_log_acell(acell_log_rate)
  GET car
  GET SD
  SET logging = true
  OPEN log_file FOR writing
  WRITE car TO log_file
  WRITE SD TO log_file
  SET last_gps_time = 0
  SET last_acell_time = 0
  WHILE logging = true
    GET time
    IF (time - last_gps_time) > gps_log_rate THEN
      GET current_gps
      WRITE current_gps TO log_file
      SET last_gps_time = time
    ELSE
      Sleep for 1 ms
    ENDIF
    IF (time - last_acell_time) > acell_log_rate THEN
      GET current_acell
      WRITE current_acell TO log_file
      SET last_acell_time = time
    ELSE
      Sleep for 1 ms
    ENDIF
    # trip ended button pressed
    IF trip has ended THEN
      SET logging = false
    ENDIF
  ENDWHILE
  CLOSE log_file
  # there will be a sepreat demon that will check if there is a
  # new trip and if there is it will uplod it to the server
END
```

Algorithm 3

```
# this psudocode decribs the login how the autnticaon works
```

```
BEGIN login
  GET username # entered
  GET password # entered
  SET password_hash = get_password_hash(username)
  IF hash(password) = password_hash THEN
    SET user_jwt = generate_jwt(username)
  ELSE
    SET user_jwt = None
  ENDIF
  RETURN user_jwt
END login

BEGIN get_protected_page (jwt)
  SET return_value = "you need to authenticate"
  IF check_if_jwt_is_valid(jwt) THEN
    GET protected_page
    SET return_value = protected_page
  ENDIF
  RETURN return_value
END get_protected_page (jwt)

BEGIN main
  GET jwt
  IF jwt = "None" THEN
    SET jwt = login()
  ENDIF
  get_protected_page(jwt)
END main
```